

MCP Server from First Principles

1. The Problem We Already Know: Tool Calling

Suppose we have an AI application and the user asks:

```
What is the weather in Delhi?
```

An LLM does not inherently know the current weather. To get live information, we can provide it with a tool:

```
@Tool
public String getWeather(String city) {
    // Call weather API
}
```

The flow becomes:

```
User
↓
LLM
↓
Chooses getWeather()
↓
Application executes the tool
↓
Weather API
↓
Result
↓
LLM
↓
Final response
```

For a single application, this architecture works perfectly well.

The problem appears when the same capabilities need to be used by many different AI applications.

2. The Integration Problem

Imagine our system provides five capabilities:

```
getCourseDetails()
searchVideos()
createStudentTicket()
getStudentProgress()
scheduleMentorshipSession()
```

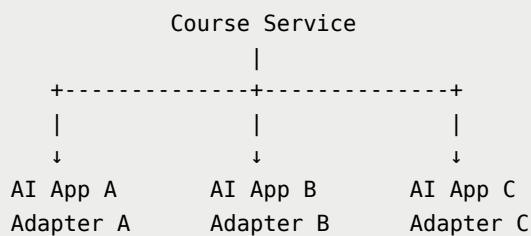
One AI application can integrate these tools directly.

But now imagine we have:

```
Coder Army Chatbot
Internal Teacher Assistant
VS Code AI Assistant
Desktop AI Assistant
Support Agent
CLI Agent
```

Each application wants to use the same capabilities.

Without a standard interface, each application needs its own integration:



Every application needs to understand:

- What capabilities are available?
- Where are they?
- What parameters do they need?
- How should they be called?
- What do they return?
- How should authentication work?

This creates an integration problem.

3. Why Standards Matter

This is not really a new AI problem. It is a common software engineering problem.

Imagine every hardware device requiring a completely different type of connector:

Keyboard	→ Connector A
Mouse	→ Connector B
Hard Drive	→ Connector C
Camera	→ Connector D

A common standard makes interaction much simpler.

We already use this idea everywhere:

USB
HTTP
JDBC
SQL
REST
SMTP

A standard does not necessarily provide a new capability.

It provides a **common way to access a capability**.

MCP applies the same idea to AI applications.

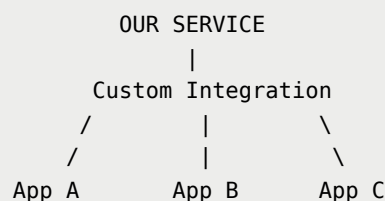
4. What Is MCP?

MCP stands for Model Context Protocol.

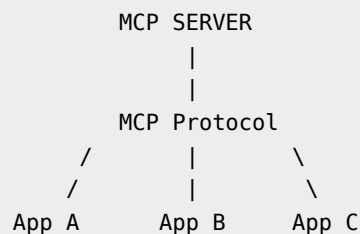
At a simple level:

MCP is a standard protocol through which AI applications can connect to external tools and data.

Instead of this:



we can expose capabilities through an MCP Server:



The important idea is **standardization**.

5. What Is an MCP Server?

An MCP Server is a program that exposes capabilities through the MCP protocol.

For example, a Task MCP Server could expose:

```
createTask
listTasks
```

completeTask

Another application can discover those capabilities.

Conceptually, it can ask:

What tools do you provide?

The MCP Server can describe a tool like:

```
{
  "name": "createTask",
  "description": "Creates a new task",
  "inputSchema": {
    "title": "string",
    "description": "string"
  }
}
```

The client now understands:

```
Tool Name
  ↓
createTask

Purpose
  ↓
Creates a task

Required Inputs
  ↓
title + description
```

It can later invoke that capability:

```
createTask

title = "Record MCP Video"
description = "Record next FDE lecture"
```

The MCP Server executes the underlying business logic.

6. MCP Server Does Not Mean LLM

An important distinction:

An MCP Server does not need an LLM.

It does not necessarily need:

```
OpenAI
Claude
Gemini
ChatClient
```

It could simply contain normal business logic:

```
public Task createTask(String title) {
    Task task = new Task(title);
    tasks.add(task);
    return task;
}
```

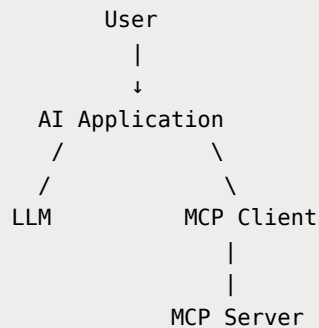
MCP simply makes that capability available to other applications through a standard protocol.

7. Where Does the LLM Come In?

The LLM normally lives inside an AI application.

```
User
↓
AI Application
↓
LLM
```

The AI application can also contain an MCP Client:



Suppose the MCP Server exposes:

```
createTask  
  
listTasks  
  
completeTask
```

The AI application discovers those capabilities and makes them available to the LLM.

The user says:

```
Add "Record Docker Lecture" to my tasks.
```

The model can decide:

```
I should use createTask.
```

and produce:

```
createTask(  
    title = "Record Docker Lecture"  
)
```

The AI application then uses MCP to execute the tool.

8. MCP vs Tool Calling

This is one of the most important distinctions.

What does Tool Calling solve?

Suppose the LLM knows about:

```
getWeather(city)
```

The user asks:

```
What is the weather in Delhi?
```

The model sees a tool definition similar to:

```
{
  "name": "getWeather",
  "description": "Get current weather",
  "parameters": {
    "city": "string"
  }
}
```

It can then request:

```
{
  "tool": "getWeather",
  "arguments": {
    "city": "Delhi"
  }
}
```

The LLM itself does not execute the function.

It essentially says:

```
Please execute getWeather("Delhi").
```

The application performs the actual execution.

Therefore:

Tool Calling answers: Which capability should the model use, and with what arguments?

What does MCP solve?

MCP solves a different problem:

How can an AI application discover and communicate with externally provided capabilities through a standard protocol?

So:

TOOL CALLING

Which tool should the model use?

while:

MCP

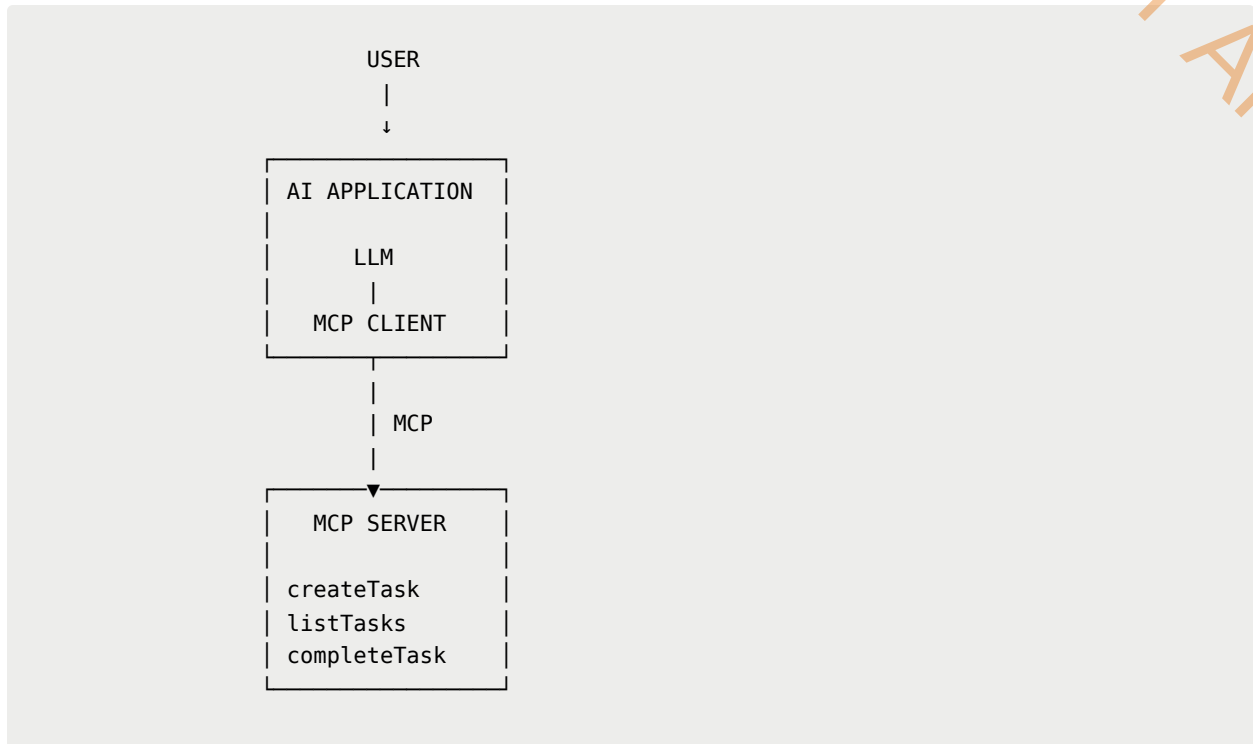
Where do those tools come from,
and how does the application access them?

These are complementary concepts.

MCP does not replace Tool Calling.

9. MCP + Tool Calling Together

The complete architecture looks like this:



The sequence is:

```

MCP Server exposes tools
↓
MCP Client discovers tools
↓
AI application gives tools to LLM
↓
LLM chooses a tool
↓
Application invokes it through MCP
↓
MCP Server executes the operation
↓
Result goes back to LLM
↓
LLM generates final response
  
```

This is where **MCP and Tool Calling** meet.

10. Tool Discovery

Suppose the AI application connects to a Task MCP Server.

The MCP Client asks:

```
What tools do you have?
```

The server responds:

```
createTask  
  
listTasks  
  
completeTask
```

This process is called **Tool Discovery**.

The MCP operation for discovering tools is:

```
tools/list
```

The server does not return only the names.

It can return definitions such as:

```
{  
  "name": "createTask",  
  "description": "Creates a new task",  
  "inputSchema": {  
    "type": "object",  
    "properties": {  
      "title": {  
        "type": "string"  
      }  
    },  
    "required": ["title"]  
  }  
}
```

The client now knows:

NAME
DESCRIPTION
INPUT SCHEMA

These definitions can then be made available to the LLM.

11. Connection with Structured Output

Tool schemas should look familiar if we already understand Structured Output.

With Structured Output:

```
LLM
↓
Schema
↓
Predictable response
```

With Tool Calling:

```
Tool
↓
Input Schema
↓
Predictable arguments
```

For example:

```
createTask
```

expects:

```
{
  "title": "string"
}
```

So when the user says:

```
Create a task called Record MCP Video.
```

the model knows that it should produce something conceptually similar to:

```
createTask(  
  title = "Record MCP Video"  
)
```

12. Executing the Tool Through MCP

Once the LLM decides to call:

```
createTask(...)
```

the Host uses the MCP Client.

The flow becomes:

```
LLM  
|  
| createTask(...)  
↓  
HOST  
|  
↓  
MCP CLIENT  
|  
| MCP Request  
↓  
MCP SERVER  
|  
↓  
createTask(...)
```

The client invokes the tool through:

```
tools/call
```

Conceptually:

```
{
  "name": "createTask",
  "arguments": {
    "title": "Record MCP Video"
  }
}
```

The MCP Server executes the underlying business logic and sends the result back.

Example:

```
{
  "id": 42,
  "title": "Record MCP Video",
  "status": "PENDING"
}
```

The result travels back:

```
MCP Server
  ↓
MCP Client
  ↓
AI Application
  ↓
LLM
```

The LLM can then generate:

```
I've added "Record MCP Video" to your tasks.
```

13. Why Not Just Use REST?

MCP does not make REST obsolete.

Suppose we already have:

```
POST /tasks
GET /tasks
PUT /tasks/{id}
```

An MCP Server can use that existing REST API internally:

```
AI Application
  |
  ↓
MCP Client
  |
  | MCP
  ↓
MCP Server
  |
  | REST
  ↓
Task Service
  |
  ↓
Database
```

REST and MCP solve different integration problems.

A REST API primarily exposes:

```
HTTP endpoints
```

An MCP Server exposes AI-oriented concepts such as:

```
Tools
Tool descriptions
Input schemas
```

Resources
Prompts
Capabilities

They can work together.

14. The $N \times M$ Integration Problem

Suppose we have:

5 AI applications

and:

10 external systems

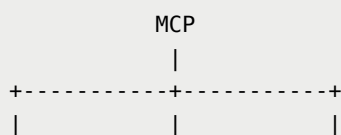
Without a shared standard, we may need many individual integrations:

App A → GitHub
App A → Slack
App A → Jira
App A → Database
App A → Files

App B → GitHub
App B → Slack
App B → Jira

...

With a common protocol:



GitHub
Server

Slack
Server

Jira
Server

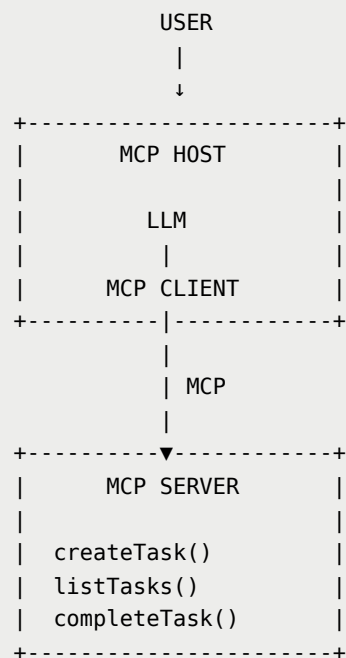
Clients understand MCP.

Servers expose their capabilities through MCP.

This creates a cleaner integration boundary.

15. MCP Architecture — Host, Client and Server

The core architecture consists of three important components:



MCP Host

The **Host** is the main AI application the user interacts with.

Examples could include:

```
Claude Code
Cursor
VS Code AI application
Desktop AI application
Our own Spring AI application
```

The Host manages the overall AI experience.

Its responsibilities can include:

```
Receive user input
  ↓
Send input + available tools to the LLM
  ↓
Receive a tool request from the LLM
  ↓
Use the MCP Client
  ↓
Call the MCP Server
  ↓
Receive the result
  ↓
Send result back to the LLM
  ↓
Display final response
```

The Host acts as the **orchestrator**.

MCP Client

The MCP Client is the software component inside the Host that knows how to communicate using MCP.

A useful comparison is JDBC:

```
Application
|
JDBC Driver
```

|
Database

Similarly:

AI Application
|
MCP Client
|
MCP Server

The MCP Client knows how to perform operations such as:

Discover tools

Invoke tools

Read resources

Retrieve prompts

MCP Server

The MCP Server is the **capability provider**.

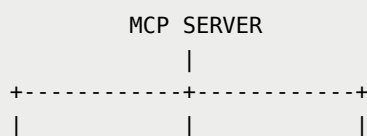
For example:

createTask()

listTasks()

completeTask()

Internally, those capabilities could communicate with almost anything:



↓ ↓ ↓
Database REST API Filesystem

They could also interact with:

GitHub
Google Calendar
Slack
Internal services
Databases
Files

The MCP Server exposes only the capabilities and context it wants clients to access.

16. MCP Server Is Not Necessarily a Microservice

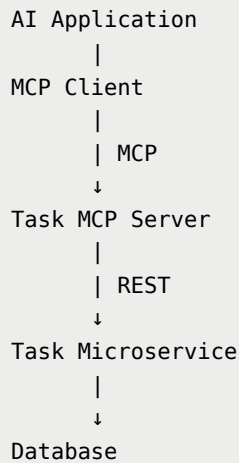
A traditional microservice may expose:

POST /tasks
GET /tasks
PUT /tasks/{id}

An MCP Server exposes MCP primitives such as:

Tools
Resources
Prompts

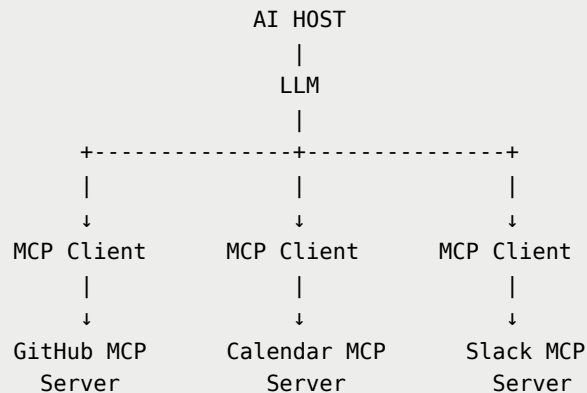
The two can coexist:



The MCP Server can act as an AI-friendly layer over an existing system.

17. One Host Can Connect to Multiple MCP Servers

An AI Host is not restricted to one MCP Server.



The model could then have capabilities such as:

```

createGitHubIssue()

scheduleMeeting()
  
```

```
sendSlackMessage()
```

without all of those implementations being built directly inside the AI application.

18. What Can an MCP Server Expose?

The three important MCP primitives are:

MCP SERVER

- └ Tools
- └ Resources
- └ Prompts

A simple mental model is:

I want AI to DO something

↓

Tools

I want AI to KNOW something

↓

Resources

I want reusable instructions

↓

Prompts

19. Tools

A **Tool** represents an executable capability.

Examples:

```
createTask()
```

```
completeTask()

listTasks()

deleteTask()
```

A tool means:

| Here is an operation that can be invoked.

A tool can contain:

```
Name

Description

Input Schema
```

For example:

```
{
  "name": "createTask",
  "description": "Creates a new task",
  "inputSchema": {
    "type": "object",
    "properties": {
      "title": {
        "type": "string"
      },
      "dueDate": {
        "type": "string"
      }
    },
    "required": ["title"]
  }
}
```

This tells the client and model:

```
title
→ required string
```

```
dueDate
→ optional string
```

Tools are generally used when something needs to be **executed or queried**.

20. Resources

Sometimes we do not want to execute an operation. We simply want to expose information.

Examples:

```
company-policy.txt
employee-handbook.pdf
application configuration
repository information
customer profile
database record
```

These can be represented as **Resources**.

A Resource is:

| Addressable context or data exposed by an MCP Server.

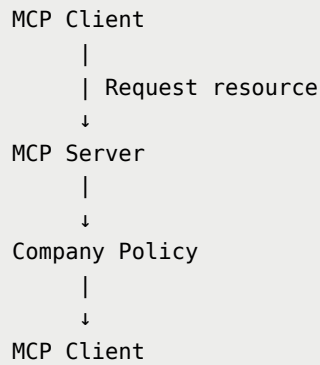
Examples:

```
company://policies/return-policy
```

or:

```
file:///project/README.md
```

Conceptually:



21. Tool vs Resource

There can be overlap.

For example:

```
getReturnPolicy()
```

could be implemented as a Tool.

But:

```
company://policy/returns
```

represents the policy itself as addressable content.

A useful distinction is:

```

TOOL
"What operation can I perform?"

RESOURCE
"What information can I access?"
  
```

22. MCP Resources vs RAG

MCP Resources do **not** replace RAG.

RAG solves a retrieval problem:

```
Documents
↓
Chunk
↓
Embed
↓
Vector Database
↓
Semantic Search
↓
Relevant Chunks
↓
LLM
```

MCP Resources expose addressable content:

```
MCP Server
↓
Expose Content
↓
Client Retrieves Content
```

They solve different problems.

They can also be combined.

For example:

```
MCP SERVER
|
searchKnowledge
|
RAG System
|
Vector Database
```

`searchKnowledge` could itself be an MCP Tool backed by a RAG pipeline.

23. Prompts

An MCP Server can also expose **reusable prompt templates or workflows**.

For example, a GitHub MCP Server could provide:

```
review_pull_request  
summarize_issue  
prepare_release_notes
```

A reusable prompt might conceptually look like:

```
review_pull_request(prNumber)  
  
↓  
  
"You are reviewing pull request {prNumber}.  
Check for bugs, maintainability issues..."
```

The client can discover these prompts and retrieve them when required.

Common prompt operations include:

```
prompts/list  
prompts/get
```

24. A Useful Control Model

A helpful conceptual distinction is:

Primitive	Typically controlled by
Tools	Model
Resources	Application
Prompts	User

Tools — Model Controlled

The user says:

```
Create a task to record tomorrow.
```

The model decides:

```
I should invoke createTask.
```

Resources — Application Controlled

The application may know that the user currently has:

```
README.md
```

open and decide to add that file as context.

Prompts — User Controlled

The UI may expose:

```
/review-code
```

and the user explicitly selects it.

This is a useful conceptual model rather than a strict limitation on every implementation.

25. How Tool Discovery Works

Suppose our application connects to a Task MCP Server.

The client asks:

What tools do you have?

The server responds:

```
createTask  
  
listTasks  
  
completeTask
```

The MCP operation used for this is:

```
tools/list
```

A response may conceptually contain:

```
{  
  "tools": [  
    {  
      "name": "createTask",  
      "description": "Creates a new task",  
      "inputSchema": {  
        "type": "object",  
        "properties": {  
          "title": {  
            "type": "string"  
          }  
        },  
        "required": ["title"]  
      }  
    },  
    {  
      "name": "listTasks",  
      "description": "Returns all tasks",  
      "inputSchema": {  
        "type": "object"  
      }  
    },  
    {  
      "name": "completeTask",  
      "description": "Marks a task as completed",  
      "inputSchema": {  
        "type": "object",  
        "properties": {  
          "taskId": {  
            "type": "string"  
          }  
        },  
        "required": ["taskId"]  
      }  
    }  
  ]  
}
```

```

    "properties": {
      "id": {
        "type": "integer"
      }
    },
    "required": ["id"]
  }
}
]
}

```

The client now understands the available tools.

26. From Discovery to LLM Tool Calling

Suppose the user asks:

```
Create a task called Record MCP Video.
```

The LLM effectively receives:

USER MESSAGE

```
Create a task called Record MCP Video.
```

AVAILABLE TOOLS

```
createTask(title)
Creates a new task.
```

```
listTasks()
Returns existing tasks.
```

```
completeTask(id)
Marks a task as complete.
```

The model can infer:

```

User wants something created
↓
createTask matches the request

```

↓
createTask requires title

It then produces a tool request:

```
createTask(  
  title = "Record MCP Video"  
)
```

This step is **Tool Calling**.

27. MCP Takes Over After Tool Selection

Once the LLM has chosen `createTask`, the Host knows that this capability comes from the MCP Server.

```
LLM  
|  
| createTask(...)  
↓  
HOST  
|  
↓  
MCP CLIENT
```

The MCP Client performs:

```
tools/call
```

Conceptually:

```
{  
  "name": "createTask",  
  "arguments": {  
    "title": "Record MCP Video"  
  }  
}
```

The MCP Server receives the request and executes:

```
createTask("Record MCP Video");
```

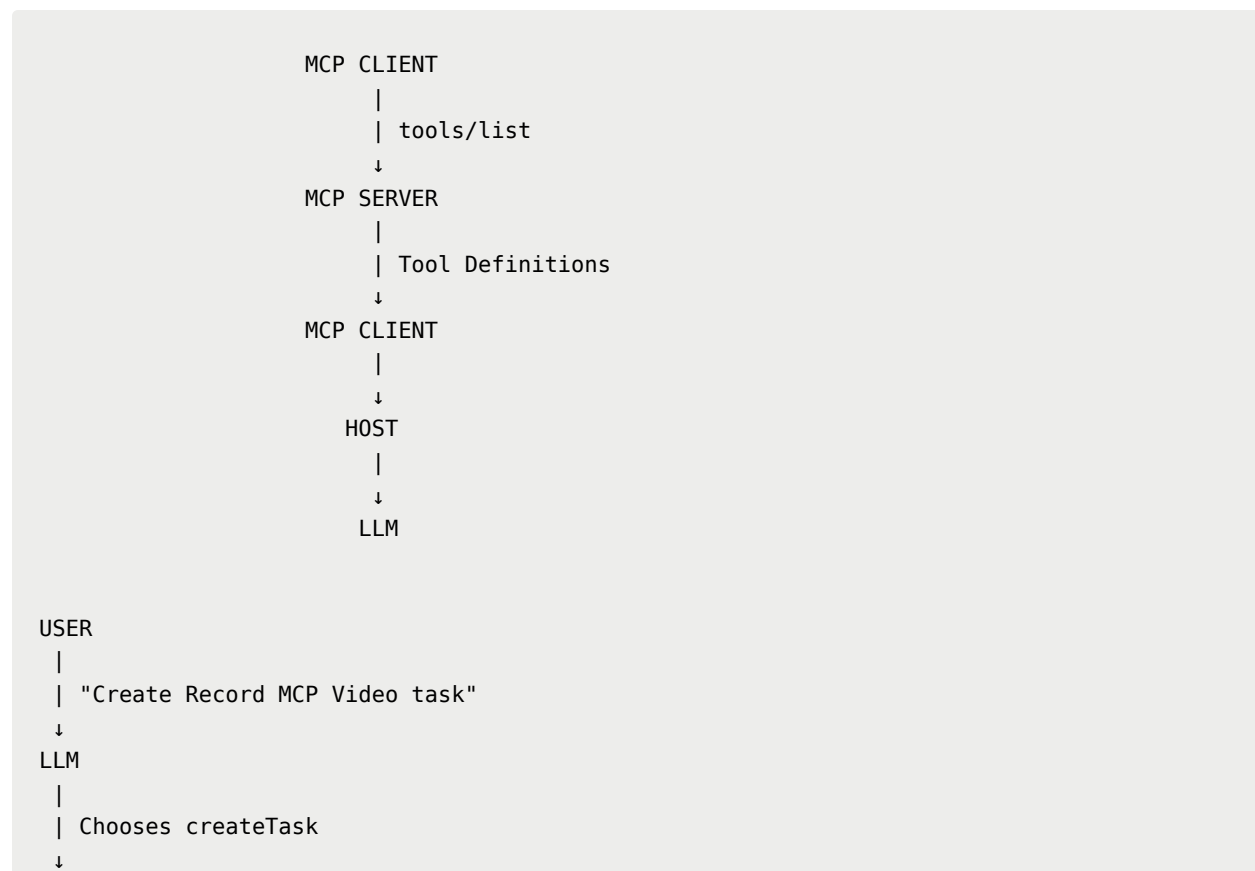
It might create:

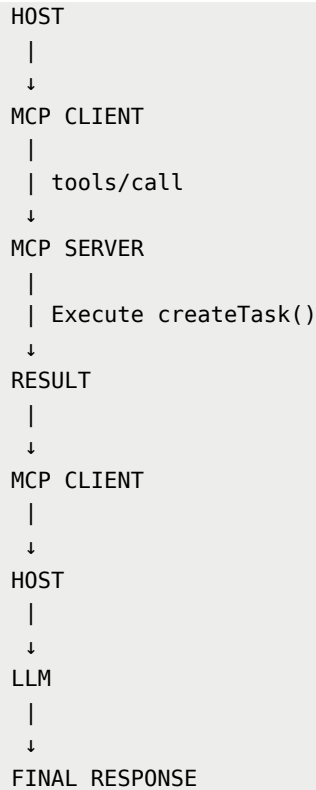
```
Task {  
  id: 1  
  title: "Record MCP Video"  
  status: PENDING  
}
```

The result returns to the MCP Client and then to the LLM.

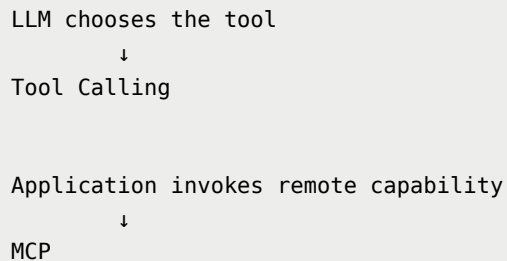
The LLM converts that result into a natural-language response for the user.

28. Complete MCP Tool Lifecycle





The key distinction remains:



29. Dynamic Tool Discovery

One major advantage of discovery is that the AI application does not necessarily need every remote capability hardcoded into its own codebase.

Suppose the MCP Server initially exposes:

```
createTask
```

```
listTasks
```

Later, the server adds:

```
completeTask
```

The updated capability can appear in the server's advertised tool list.

The AI application does not necessarily need a manually implemented local:

```
completeTask(...)
```

method.

This is one of the important benefits of MCP-based capability discovery.

30. Security Still Belongs to the Host

Discovering a tool does not mean the model should automatically be allowed to execute everything.

The Host can still apply:

```
Permissions
```

```
User Approval
```

```
Tool Filtering
```

```
Authentication
```

```
Security Policies
```

For example:

`readFile`

might be allowed automatically.

But:

`deleteRepository`

could require explicit user confirmation.

MCP standardizes communication.

It does **not** remove the application's responsibility for security and authorization.

31. Does `tools/list` Run for Every User Message?

Not necessarily.

If the available tool catalogue has not changed, the Host or MCP Client can reuse previously discovered information.

Conceptually:

```
Discover tools
  ↓
Cache available definitions
  ↓
Reuse when appropriate
```

Operations such as:

```
tools/list
resources/list
prompts/list
```

represent discovery of these capabilities.

32. Practical Demo Architecture

To understand MCP clearly, build **two separate applications**.

1. TASK MCP SERVER

No LLM
No OpenAI

Exposes:
createTask
listTasks
completeTask

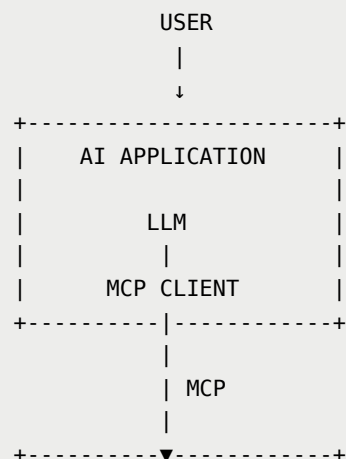
and:

2. AI APPLICATION

Contains the LLM
Contains an MCP Client

Connects to Task MCP Server
Discovers its capabilities
Makes those tools available to the LLM

Architecture:



```

| TASK MCP SERVER |
| createTask      |
| listTasks       |
| completeTask    |
+-----+

```

This separation makes one thing very clear:

```

MCP SERVER
≠
LLM Application

```

The MCP Server provides capabilities.

The AI application contains the intelligence that decides when those capabilities should be used.

Final Mental Model

The easiest way to remember everything is:

```

Tool Calling
=
The LLM decides which capability to use.

MCP
=
The AI application gets access to external
capabilities through a standard protocol.

```

And the complete flow is:

```

MCP Server exposes capabilities
↓
MCP Client discovers capabilities
↓
Host gives relevant tools to LLM
↓
LLM selects a tool

```

↓
MCP Client invokes it
↓
MCP Server executes it
↓
Result returns to the LLM
↓
User gets the final response

Coder Army

Key Takeaways

- **MCP stands for Model Context Protocol.**
- MCP provides a standard way for AI applications to connect with external capabilities and context.
- An **MCP Server does not need an LLM.**
- The **Host** manages the AI experience.
- The **MCP Client** communicates using MCP.
- The **MCP Server** exposes capabilities.
- MCP Servers can expose **Tools, Resources, and Prompts.**
- `tools/list` is used for tool discovery.
- `tools/call` is used to invoke a tool.
- **Tool Calling and MCP are complementary**, not competing concepts.
- Tool Calling decides **what capability to use.**
- MCP standardizes **how the application accesses that external capability.**
- MCP can coexist with REST APIs, databases, RAG systems, files, and existing backend services.
- The real benefit becomes clear when the same external capabilities need to be reused by multiple AI applications.